

Deployment of Zomato Clone Application

End-to-End DevOps Deployment with CI/CD, Kubernetes and Monitoring

Name: Pooja Prakash Belgaumkar

- This project demonstrates a complete DevOps lifecycle for deploying a Zomato web application. It covers containerization, CI/CD automation, Kubernetes orchestration, security scanning, email notifications and real-time monitoring, all implemented on AWS cloud infrastructure.

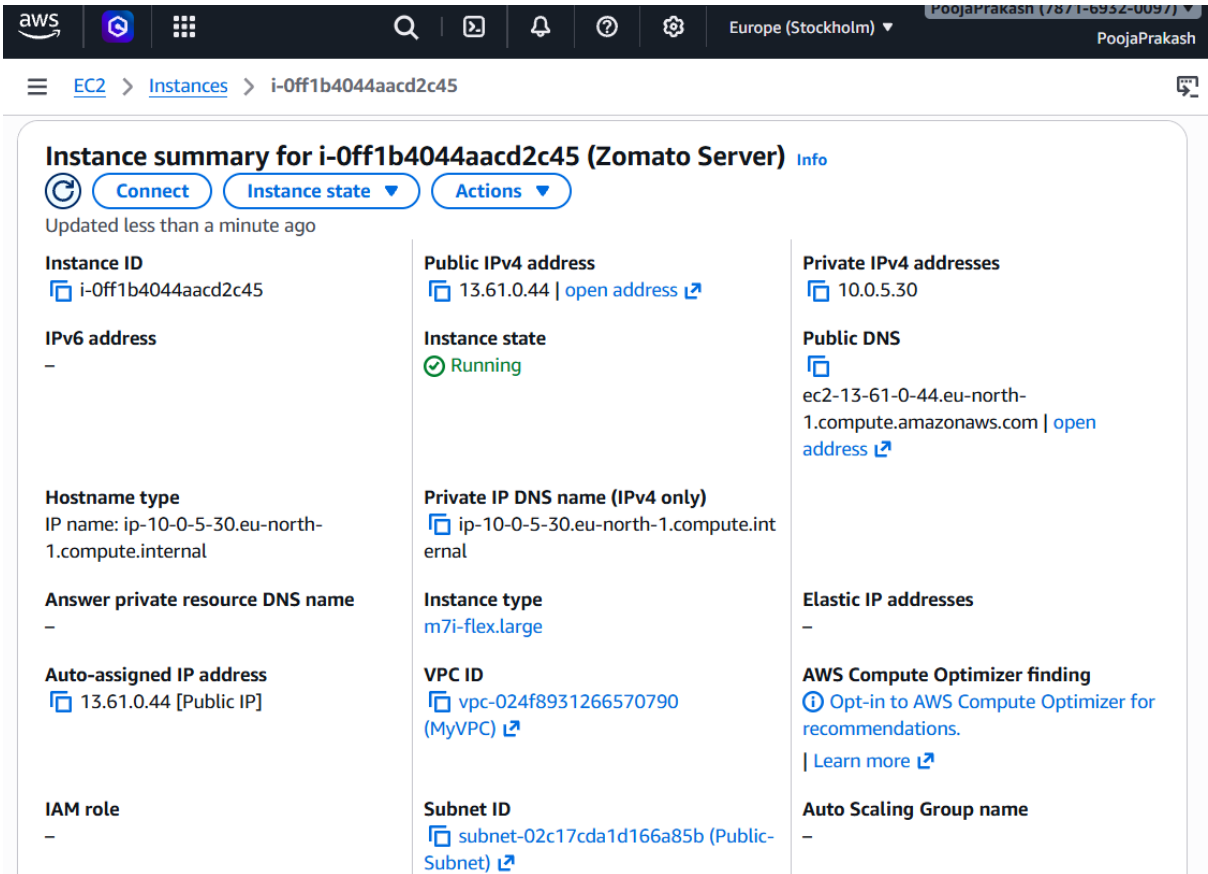
The screenshot shows a GitHub repository page for 'DevOps-Project-Zomato-Kastro' by user 'PoojaPrakash08'. The repository is public and forked from 'KastroVKiran/DevOps-Project-Zomato-Kastro'. The page displays the repository's structure, including folders like 'Kubernetes', 'public', and 'src', and files like '.gitignore', 'DevOps Project - Zomato - Kastro.txt', 'Dockerfile', 'README.md', 'jenkinsfile', 'package-lock.json', and 'package.json'. The repository has 1 branch (master) and 0 tags. The commit history shows 22 commits, with the most recent commit by 'KastroVKiran' updating the 'DevOps Project - Zomato - Kastro.txt' file 4 months ago.

File/Folder	Commit Message	Commit Date
Kubernetes	Update service.yaml	2 years ago
public	Zomato Project by Kastro	2 years ago
src	Zomato Project by Kastro	2 years ago
.gitignore	Zomato Project by Kastro	2 years ago
DevOps Project - Zomato - Kastro.txt	Update DevOps Project - Zomato - Kastro.txt	4 months ago
Dockerfile	Zomato Project by Kastro	2 years ago
README.md	Update README.md	2 years ago
jenkinsfile	Update jenkinsfile	2 years ago
package-lock.json	Zomato Project by Kastro	2 years ago
package.json	Zomato Project by Kastro	2 years ago

Phase 1: Docker Containerization and CI/CD Pipeline

1. Infrastructure Setup

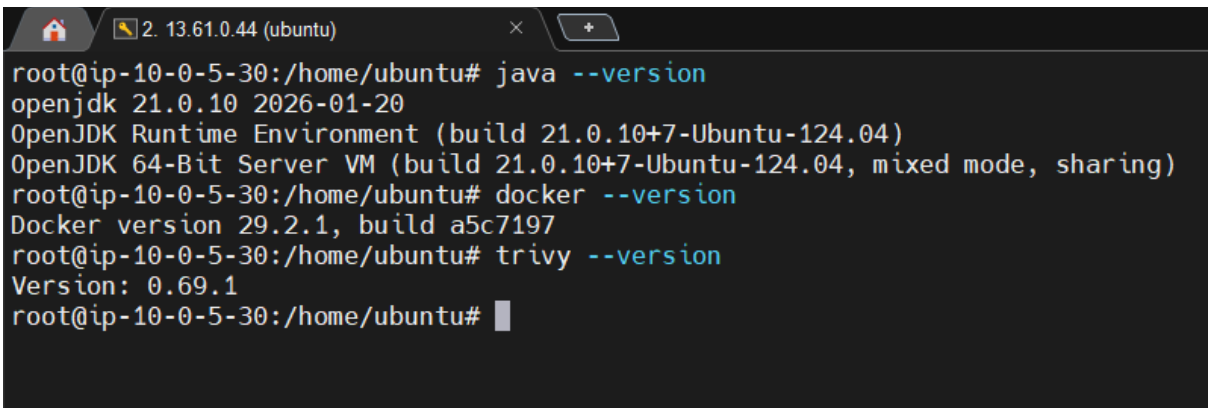
- Launched an Ubuntu EC2 instance named **Zomato Server** with a large instance type and 30 GB storage.



The screenshot displays the AWS Management Console interface for an EC2 instance. The top navigation bar shows the AWS logo, a search bar, and the user's profile (PoojaPrakash). The breadcrumb trail indicates the path: EC2 > Instances > i-Off1b4044aacd2c45. The main content area is titled "Instance summary for i-Off1b4044aacd2c45 (Zomato Server)" and includes buttons for "Connect", "Instance state", and "Actions". The instance is shown as "Running". The summary is organized into three columns:

Instance ID	Public IPv4 address	Private IPv4 addresses
i-Off1b4044aacd2c45	13.61.0.44 open address	10.0.5.30
IPv6 address	Instance state	Public DNS
-	Running	ec2-13-61-0-44.eu-north-1.compute.amazonaws.com open address
Hostname type	Private IP DNS name (IPv4 only)	Elastic IP addresses
IP name: ip-10-0-5-30.eu-north-1.compute.internal	ip-10-0-5-30.eu-north-1.compute.internal	-
Answer private resource DNS name	Instance type	AWS Compute Optimizer finding
-	m7i-flex.large	Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address	VPC ID	Auto Scaling Group name
13.61.0.44 [Public IP]	vpc-024f8931266570790 (MyVPC)	-
IAM role	Subnet ID	
-	subnet-02c17cda1d166a85b (Public-Subnet)	

- Connect to the instance and Installed java, Docker, Trivy, AWS CLI and Jenkins



```
root@ip-10-0-5-30:/home/ubuntu# java --version
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
root@ip-10-0-5-30:/home/ubuntu# docker --version
Docker version 29.2.1, build a5c7197
root@ip-10-0-5-30:/home/ubuntu# trivy --version
Version: 0.69.1
root@ip-10-0-5-30:/home/ubuntu#
```

```

root@ip-10-0-5-30:/home/ubuntu# aws --version
aws-cli/2.33.28 Python/3.13.11 Linux/6.14.0-1018-aws exe/x86_64.ubuntu.24
root@ip-10-0-5-30:/home/ubuntu# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 09:34:00 UTC; 5min ago
     Main PID: 5284 (java)
        Tasks: 43 (limit: 9283)
      Memory: 882.3M (peak: 882.6M)
         CPU: 30.792s
        CGroup: /system.slice/jenkins.service
                └─5284 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/

Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.888+0000 [id=232] INFO jenkins
Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.888+0000 [id=232] INFO jenkins
Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.904+0000 [id=232] INFO jenkins
Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.905+0000 [id=232] INFO jenkins
Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.988+0000 [id=230] INFO jenkins
Feb 24 09:37:19 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:19.988+0000 [id=230] INFO jenkins
Feb 24 09:37:20 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:20.008+0000 [id=231] INFO jenkins
Feb 24 09:37:20 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:20.011+0000 [id=230] INFO jenkins
Feb 24 09:37:20 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:20.234+0000 [id=230] INFO jenkins
Feb 24 09:37:20 ip-10-0-5-30 jenkins[5284]: 2026-02-24 09:37:20.235+0000 [id=85] INFO h.m.Upd
lines 1-20/20 (END)

```

- Logged in to Docker Hub and installed Docker Scout.

```

root@ip-10-0-5-30:/home/ubuntu# docker login -u poojaprakash08

Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-10-0-5-30:/home/ubuntu# curl -sSfL https://raw.githubusercontent.com/docker/scout-cli/main/install.s
h | sh -s -- -b /usr/local/bin
[info] fetching release script for tag='v1.19.0'
[info] using release tag='v1.19.0' version='1.19.0' os='linux' arch='amd64'
[info] installed /usr/local/bin/docker-scout
root@ip-10-0-5-30:/home/ubuntu# sudo chmod 777 /var/run/docker.sock
root@ip-10-0-5-30:/home/ubuntu#

```

- Installed SonarQube using Docker.

```

root@ip-10-0-5-30:/home/ubuntu# docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
60d98d907669: Pull complete
4f4fb700ef54: Pull complete
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
600f318704dc: Download complete
ce2bd68be4fc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
e824c1ed1ead15414a5b2a4e3a815c57c83a6f4a4ae3239dc5a5b7d6270c78ca
root@ip-10-0-5-30:/home/ubuntu# docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
sonarqube:lts-community	f709975ab31d	1.02GB	398MB	U

```

root@ip-10-0-5-30:/home/ubuntu# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
e824c1ed1ead	sonarqube:lts-community	"/opt/sonarqube/dock..."	25 seconds ago	Up 21 seconds	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp

```

root@ip-10-0-5-30:/home/ubuntu#

```

2. Jenkins Configuration

- Accessed the Jenkins web interface on port 8080 and installed required plugins.

Plugin	Status
commons-lang3 v3.x Jenkins API	Success
Icons API	Success
Folders	Success
OWASP Markup Formatter	Success
ASM API	Success
JSON Path API	Success
Structs	Success
Pipeline: Step API	Success

- Configured global tools in Jenkins: JDK, SonarQube, Nodejs, OWASP Dependency-Check, Docker tools in the Jenkins to use these tools in pipeline

JDK installations

+ Add JDK

JDK

Name:

☒ Install automatically ?

☒ Install from adoptium.net ?

Version:

+ Add Installer

SonarQube Scanner installations

+ Add SonarQube Scanner

SonarQube Scanner

Name:

☒ Install automatically ?

☒ Install from Maven Central

Version:

+ Add Installer

**Jenkins**

/ Manage Jenkins



Tools

NodeJS installations

[+ Add NodeJS](#)

NodeJS

Name

node25

☒ Install automatically

Install from nodejs.org

Version

NodeJS 25.6.1

**Jenkins**

/ Manage Jenkins



Tools

Dependency-Check installations

[+ Add Dependency-Check](#)

Dependency-Check

Name

DP-Check

☒ Install automatically

Install from github.com

Version

dependency-check 12.2.0

**Jenkins**

/ Manage Jenkins



Tools

Docker installations

[+ Add Docker](#)

Docker

Name

docker

☒ Install automatically

Download from docker.com

Docker version

latest

- Accessed SonarQube on port 9000, generated a token, and added it to Jenkins credentials.
- Added Docker Hub credentials in Jenkins.

Not secure 13.61.0.44:9000/admin/users

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration ? Search for projects... A

Administration

Configuration ▾ Security ▾ Projects ▾ System Marketplace

Users [Create User](#)

Create and administer individual users.

Search by login or name...

	SCM Accounts	Last connection	Groups	Tokens
A Administrator admin		< 1 hour ago	sonar-administrators sonar-users	0 Manage

Tokens of Administrator

Generate Tokens

Name Expires in

Enter Token Name 30 days [Generate](#)

! New token "token" has been created. Make sure you copy it now, you won't be able to see it again!

[Copy](#) squ_1e6eeb11d7ec183630f4de23335b6768ea5261c8

Name	Type	Project	Last use	Created	Expiration	
token	User		Never	February 24, 2026	March 26, 2026	Revoke

 **Jenkins** / Manage Jenkins ▾ / Credentials / System / Global

[Search](#) [Settings](#) [User](#)

Global

[+ Add Credentials](#)

Credentials that should be available everywhere.

 Sonar-token Sonar-token - Sonar-token	Settings More
 poojaprakash08/***** (docker) docker - docker	Settings More

- Created a webhook in SonarQube to notify Jenkins when source code is updated.

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration ? A

Administration

Configuration ▾ Security ▾ Projects ▾ System Marketplace

Webhooks Create

Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the [Webhooks documentation](#).

No webhook defined.

Create Webhook

All fields marked with * are required

Name *



URL *



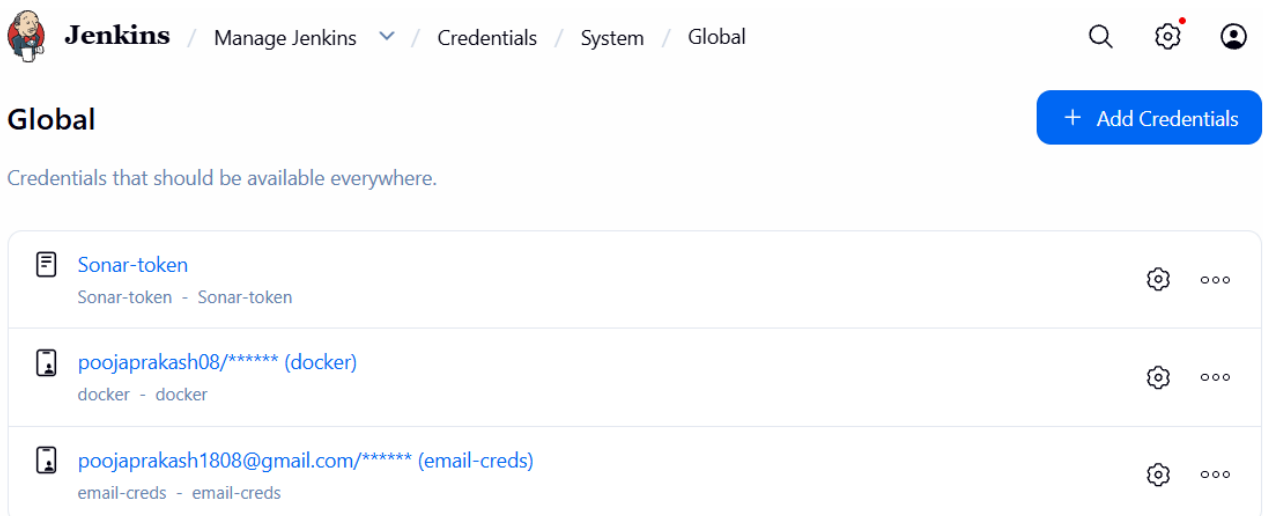
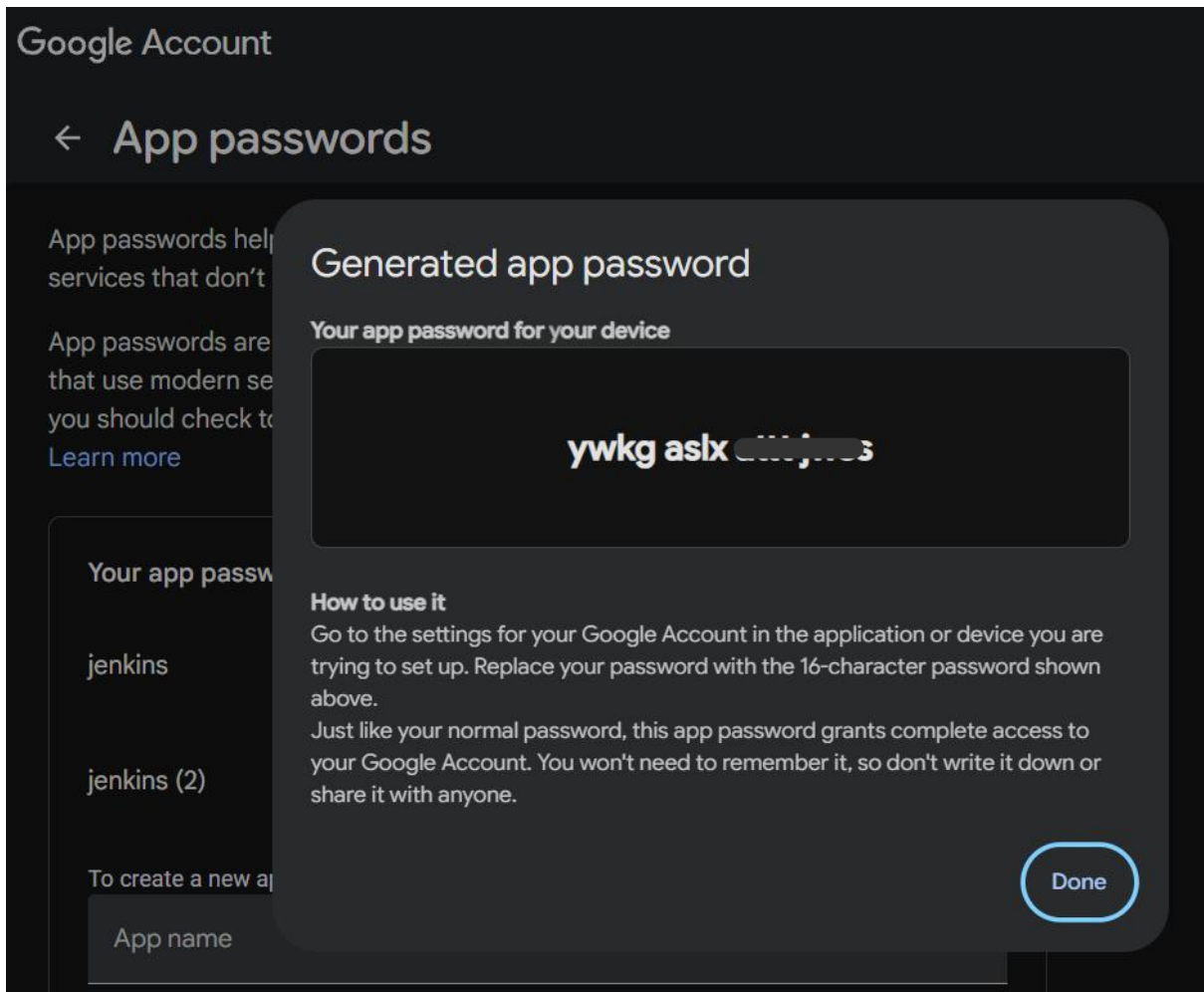
Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

CreateCancel

- Generated a Gmail App Password and added it to Jenkins credentials to enable email notifications for pipeline status.



- Completed system configuration for SonarQube and Email Notifications in Jenkins.

 **Jenkins** / Manage Jenkins ▾ / System ▾

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration

☐ Environment variables

SonarQube installations

[List of SonarQube installations](#)

Name

sonar-server

Server URL

Default is http://localhost:9000

http://13.61.0.44:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is enabled

Sonar-token

 **Jenkins** / Manage Jenkins ▾ / System ▾

Extended E-mail Notification

SMTP server

smtp.gmail.com

SMTP Port

465

Advanced ^

 Edited

Credentials

poojaprakash1808@gmail.com/***** (email-creds)

☒ Use SSL

☐ Use TLS

☒ Use OAuth 2.0



E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail suffix ?

Advanced ^

Edited

☒ Use SMTP Authentication ?

User Name

poojaprakash1808@gmail.com

For security when using authentication it is recommended to enable

Password

.....

☒ Use SSL ?



☐ Use TLS

SMTP Port ?

465

Reply-To Address

poojaprakash1808@gmail.com

Charset

UTF-8

☒ Test configuration by sending test e-mail

Test e-mail recipient

poojaprakash1808@gmail.com

Email was successfully sent

Test configuration

3. CI/CD Pipeline Execution

- Created a Jenkinsfile to automate the pipeline stages: Clone source code from GitHub, Perform SonarQube code analysis, Install npm dependencies, Run OWASP dependency check, Perform Trivy file system scan, Build Docker image, Tag and push image to Docker Hub, Run Docker Scout image analysis, Deploy containerized application and Send email notifications

```
Welcome | jenkinsfile X
jenkinsfile
1 pipeline {
2     agent any
3     tools {
4         jdk 'jdk17'
5         nodejs 'node25'
6     }
7     environment {
8         SCANNER_HOME=tool 'sonar-scanner'
9     }
10    stages {
11        stage ("clean workspace") {
12            steps {
13                cleanWs()
14            }
15        }
16        stage ("Git Checkout") {
17            steps {
18                git 'https://github.com/PoojaPrakash08/DevOps-Project-Zomato-Kastro.git'
19            }
20        }
21        stage("Sonarqube Analysis"){
22            steps{
23                withSonarQubeEnv('sonar-server') {
24                    sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=zomato \
25                        -Dsonar.projectKey=zomato '''
26                }
27            }
28        }
29        stage("Code Quality Gate"){
30            steps {
31                script {
32                    waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
33                }
34            }
35        }
36    }
37 }
```

```

36 stage("Install NPM Dependencies") {
37     steps {
38         sh "npm install"
39     }
40 }
41 stage('OWASP FS SCAN') {
42     steps {
43         dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit -n', odcInstallation: 'DP-Check'
44         dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
45     }
46 }
47 stage ("Trivy File Scan") {
48     steps {
49         sh "trivy fs . > trivy.txt"
50     }
51 }
52 stage ("Build Docker Image") {
53     steps {
54         sh "docker build -t zomato ."
55     }
56 }
57 stage ("Tag & Push to DockerHub") {
58     steps {
59         script {
60             withDockerRegistry(credentialsId: 'docker') {
61                 sh "docker tag zomato poojaprakash08/zomato:latest "
62                 sh "docker push poojaprakash08/zomato:latest "
63             }
64         }
65     }
66 }

```

```

67 stage('Docker Scout Image') {
68     steps {
69         script{
70             withDockerRegistry(credentialsId: 'docker', toolName: 'docker'){
71                 sh 'docker-scout quickview poojaprakash08/zomato:latest'
72                 sh 'docker-scout cves poojaprakash08/zomato:latest'
73                 sh 'docker-scout recommendations poojaprakash08/zomato:latest'
74             }
75         }
76     }
77 }
78 stage ("Deploy to Container") {
79     steps {
80         sh 'docker run -d --name zomato -p 3000:3000 poojaprakash08/zomato:latest'
81     }
82 }
83 }

```

```

84     post {
85     always {
86         emailx attachLog: true,
87         subject: "'${currentBuild.result}'",
88         body: ""
89             <html>
90             <body>
91                 <div style="background-color: #FFA07A; padding: 10px; margin-bottom: 10px;">
92                     <p style="color: white; font-weight: bold;">Project: ${env.JOB_NAME}</p>
93                 </div>
94                 <div style="background-color: #90EE90; padding: 10px; margin-bottom: 10px;">
95                     <p style="color: white; font-weight: bold;">Build Number: ${env.BUILD_NUMBER}</p>
96                 </div>
97                 <div style="background-color: #87CEEB; padding: 10px; margin-bottom: 10px;">
98                     <p style="color: white; font-weight: bold;">URL: ${env.BUILD_URL}</p>
99                 </div>
100             </body>
101             </html>
102         "",
103         to: 'poojaprasakash1808@gmail.com',
104         mimeType: 'text/html',
105         attachmentsPattern: 'trivy.txt'
106     }
107 }
108 }
109

```

- Created a Dockerfile to build the application image.

```

Welcome  jenkinsfile  Dockerfile X
Dockerfile
1  # Use Node.js 16 slim as the base image
2  FROM node:16-slim
3
4  # Set the working directory
5  WORKDIR /app
6
7  # Copy package.json and package-lock.json to the working directory
8  COPY package*.json ./
9
10 # Install dependencies
11 RUN npm install
12
13 # Copy the rest of the application code
14 COPY . .
15
16 # Build the React app
17 RUN npm run build
18
19 # Expose port 3000 (or the port your app is configured to listen on)
20 EXPOSE 3000
21
22 # Start your Node.js server (assuming it serves the React app)
23 CMD ["npm", "start"]
24

```

- Created a Jenkins Pipeline job named Zomato Application and executed the build.



Jenkins / All / New Item



New Item

Enter an item name

Zomato-Application

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Jenkins / Zomato-Application / Configure



Configure

General

Triggers

Pipeline

Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   tools {
4     jdk 'jdk17'
5     nodejs 'node25'
6   }
7   environment {
8     SCANNER_HOME=tool 'sonar-scanner'
9   }
10  stages {
11    stage ("clean workspace") {
12      steps {
13        cleanWs()
14      }
15    }
16  }
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

Save

Apply

Jenkins / Zomato-Application

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

SonarQube

Stages

Rename

Pipeline Syntax

Zomato-Application

Add description

Stage View

#5

Feb 24 18:27

No Changes

Declarative: Tool Install	clean workspace	Git Checkout	Sonarqube Analysis	Code Quality Gate	Install NPM Dependencies	OWASP FS SCAN	Trivy File Scan	Build Docker Image	Tag Push Docker
177ms	316ms	1s	14s	243ms	16s	5min 59s	3s	1min 6s	9s

Average stage times:

(full run time: ~10min 34s)

SonarQube Quality Gate

zomato

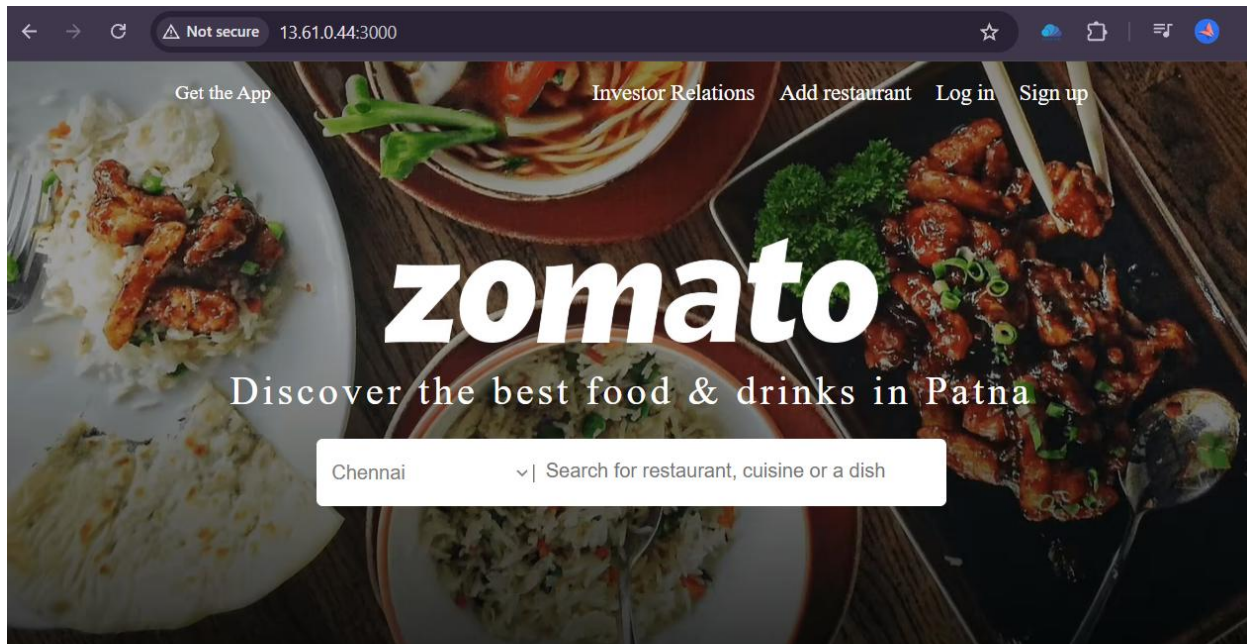
Passed

server-side processing:

Success

Latest Dependency-Check

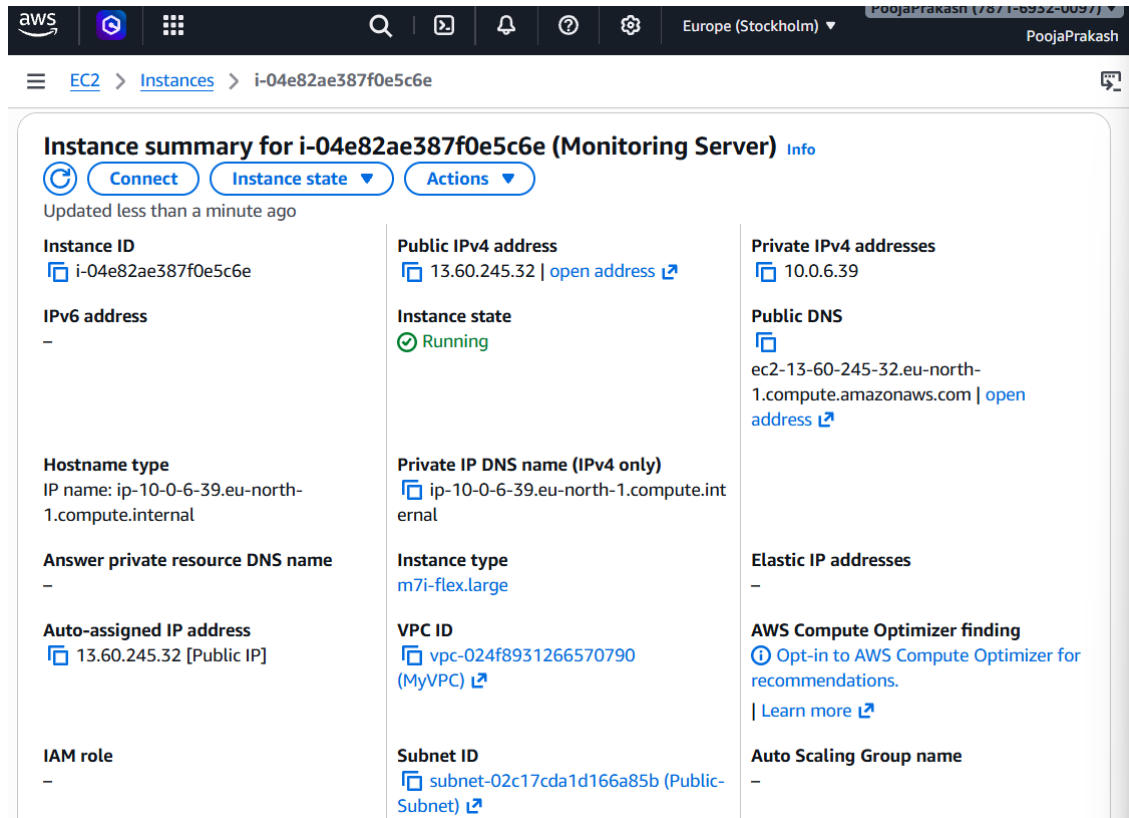
Permalinks



Phase 2: Monitoring with Prometheus and Grafana

1. Monitoring Server Setup

- Launched another Ubuntu EC2 instance named **Monitoring Server**.

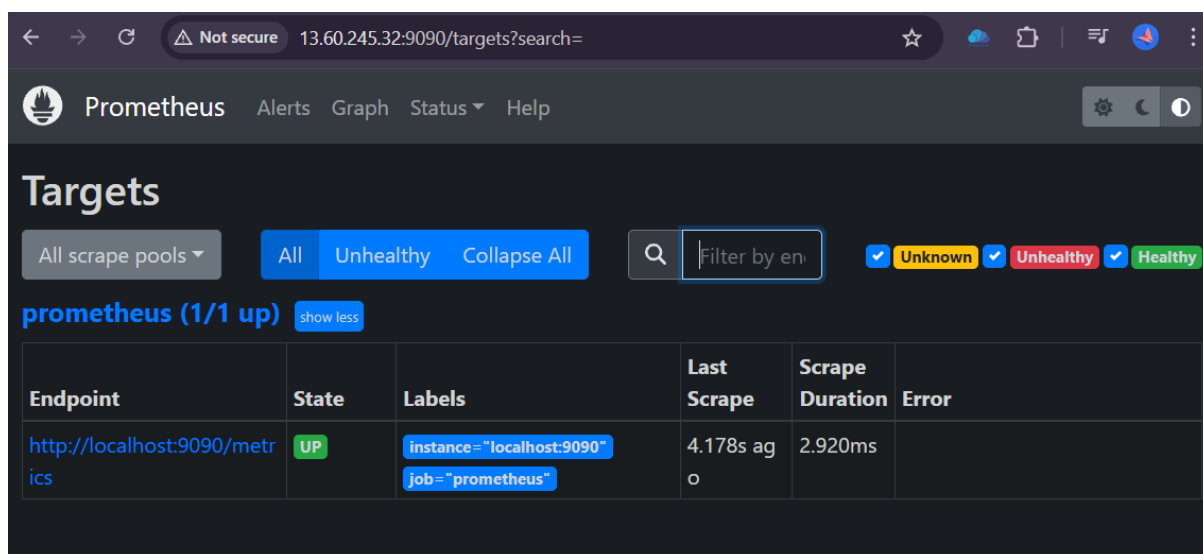


- Installed Prometheus on the Monitoring Server.

```
root@ip-10-0-6-39:/home/ubuntu/prometheus-2.47.1.linux-amd64# sudo systemctl start prometheus
root@ip-10-0-6-39:/home/ubuntu/prometheus-2.47.1.linux-amd64# sudo systemctl status prometheus
● prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 15:05:30 UTC; 13s ago
     Main PID: 1772 (prometheus)
        Tasks: 7 (limit: 9283)
      Memory: 18.7M (peak: 19.1M)
         CPU: 49ms
    CGroup: /system.slice/prometheus.service
            └─1772 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.p>

Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.170Z caller=head.go:681 level=info co>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.170Z caller=head.go:689 level=info co>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.170Z caller=head.go:760 level=info co>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.170Z caller=head.go:797 level=info co>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.171Z caller=main.go:1045 level=info f>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.172Z caller=main.go:1048 level=info m>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.172Z caller=main.go:1229 level=info m>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.173Z caller=main.go:1266 level=info m>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.173Z caller=main.go:1009 level=info m>
Feb 24 15:05:30 ip-10-0-6-39 prometheus[1772]: ts=2026-02-24T15:05:30.173Z caller=manager.go:1009 level=inf>
lines 1-20/20 (END)
```

- Accessed Prometheus on port 9090.



- Installed Node Exporter to collect system metrics.

```
root@ip-10-0-6-39:~# sudo systemctl status node_exporter
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 15:12:14 UTC; 17s ago
     Main PID: 1911 (node_exporter)
        Tasks: 5 (limit: 9283)
       Memory: 2.4M (peak: 2.6M)
          CPU: 5ms
       CGroup: /system.slice/node_exporter.service
               └─1911 /usr/local/bin/node_exporter --collector.logind

Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=node_exporter.go:117 l>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=tlscfg.go:274 leve>
Feb 24 15:12:14 ip-10-0-6-39 node_exporter[1911]: ts=2026-02-24T15:12:14.876Z caller=tlscfg.go:277 leve>
lines 1-20/20 (END)
```

2. Prometheus Configuration

- Modified the prometheus.yml file to scrape metrics from:
 - Node Exporter
 - Jenkins

```
root@ip-10-0-6-39:~# cd /etc/prometheus/
root@ip-10-0-6-39:/etc/prometheus# ls
console_libraries  consoles  prometheus.yml
root@ip-10-0-6-39:/etc/prometheus# sudo vi prometheus.yml
```

```
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['13.60.245.32:9100']

  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['13.61.0.44:8080']

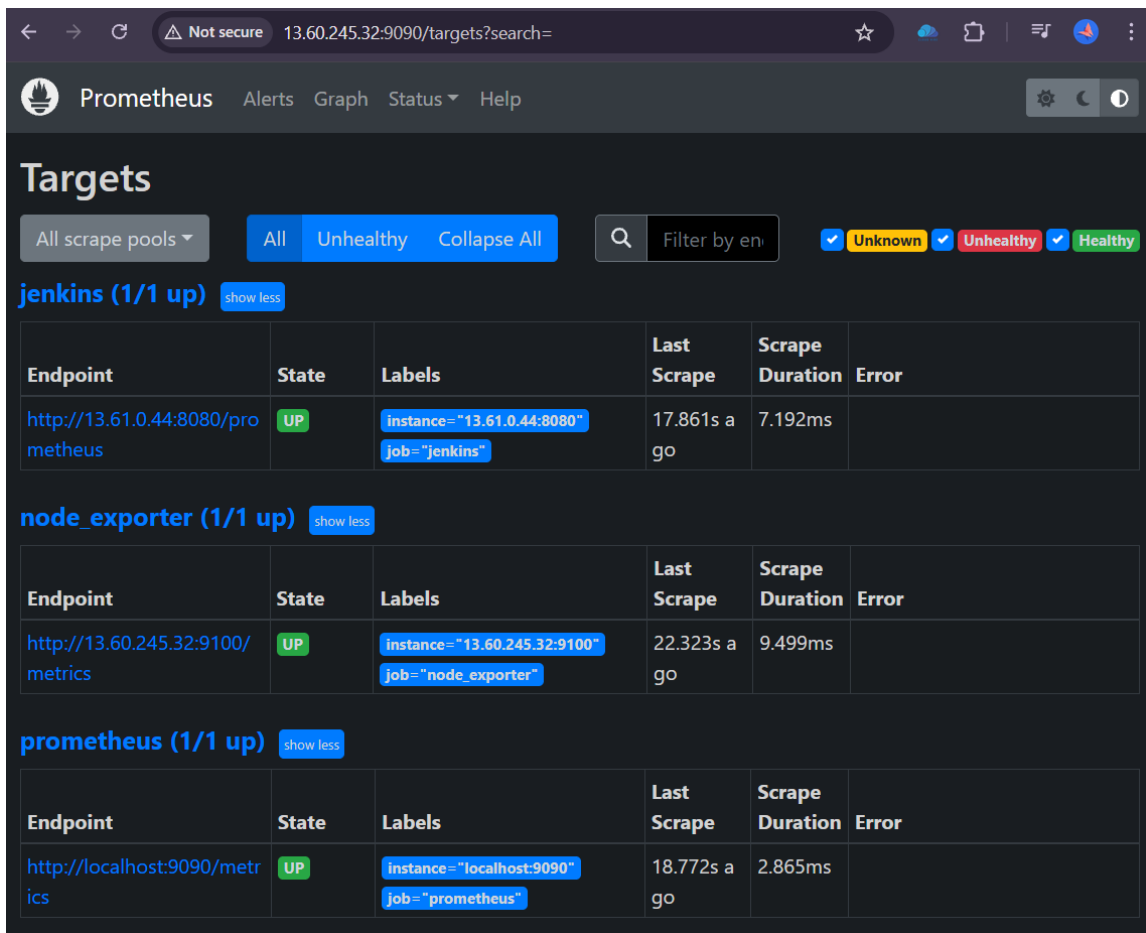
~
~
-- INSERT --
```

- Validated the configuration using promtool.

```
root@ip-10-0-6-39:/etc/prometheus# promtool check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax

root@ip-10-0-6-39:/etc/prometheus#
```

- Reloaded Prometheus and verified targets were up.



The screenshot shows the Prometheus web interface at the URL 13.60.245.32:9090/targets?search=. The page title is 'Targets'. There are tabs for 'All', 'Unhealthy', and 'Collapse All'. A search bar is present with the text 'Filter by en'. There are three status filters: 'Unknown' (checked), 'Unhealthy' (checked), and 'Healthy' (checked). The page lists three targets, each with a 'show less' button.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
jenkins (1/1 up) show less					
http://13.61.0.44:8080/prometheus	UP	instance="13.61.0.44:8080" job="jenkins"	17.861s ago	7.192ms	
node_exporter (1/1 up) show less					
http://13.60.245.32:9100/metrics	UP	instance="13.60.245.32:9100" job="node_exporter"	22.323s ago	9.499ms	
prometheus (1/1 up) show less					
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	18.772s ago	2.865ms	

3. Grafana Visualization

- Installed Grafana on the Monitoring Server.

```

root@ip-10-0-6-39:~# sudo systemctl status grafana-server
● grafana-server.service - Grafana instance
   Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 15:33:09 UTC; 17s ago
     Docs: http://docs.grafana.org
    Main PID: 3154 (grafana)
      Tasks: 10 (limit: 9283)
    Memory: 170.5M (peak: 170.9M)
       CPU: 2.934s
    CGroup: /system.slice/grafana-server.service
            └─3154 /usr/share/grafana/bin/grafana server --config=/etc/grafana/grafana.ini --pidfile=/run/

Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.backgroundinstaller t=2026-02-24T15:33:18.2035218Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.installer t=2026-02-24T15:33:18.569060092Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=installer.fs t=2026-02-24T15:33:18.625171482Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugins.registration t=2026-02-24T15:33:18.640024181Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.backgroundinstaller t=2026-02-24T15:33:18.6400585Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.backgroundinstaller t=2026-02-24T15:33:18.6400806Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.installer t=2026-02-24T15:33:18.917421523Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=installer.fs t=2026-02-24T15:33:18.970900319Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugins.registration t=2026-02-24T15:33:18.98829971Z level=info
Feb 24 15:33:18 ip-10-0-6-39 grafana[3154]: logger=plugin.backgroundinstaller t=2026-02-24T15:33:18.9883394Z level=info
lines 1-21/21 (END)

```

- Accessed Grafana on port 3000 and added Prometheus as a Data Source.

The screenshot shows the Grafana web interface for editing a Prometheus data source. The browser address bar shows the URL `13.60.245.32:3000/connections/datasources/edit/ffe7yyn8tuqrka`. The breadcrumb navigation is `Connections > Data sources > prometheus`. The main heading is `prometheus` with the type `Type: Prometheus`. Below this is a navigation bar with tabs: `Settings` (active), `Dashboards`, `Permissions`, `Insights`, and `Cache`. A message box says: `Configure your Prometheus data source below` or skip the effort and get Prometheus as fully-managed, scalable, and hosted data source from Grafana Labs with... Below this is a form with a `Name` field containing `prometheus`, a `Default` toggle switch, and a `Connection` section with a `Prometheus server URL *` field containing `http://13.60.245.32:9090`. A note states: `Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, Fields marked with * are required`.

The screenshot shows a success message in a green box: `✓ Successfully queried the Prometheus API.` Below the message is a text: `Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.` At the bottom are two buttons: `Delete` (red) and `Save & test` (blue).

- Imported official dashboards for:
 - Node Exporter
 - Jenkins
- These dashboards provided visual representation of system and pipeline metrics.

Dashboard

>

Import dashboard

Import dashboard

Import dashboard from file or Grafana.com

Upload dashboard JSON file

Drag and drop here or click to browse

Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

1860

Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_0HnEoN4z",
  "panels": [...]
  ...
}
```

Load

Cancel

Dashboard

>

Import dashboard

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](https://grafana.com)

Published by

rfmoz

Updated on

2025-09-27 22:45:03

Options

Name

Node Exporter Full

Folder

Dashboards

Unique identifier (UID)

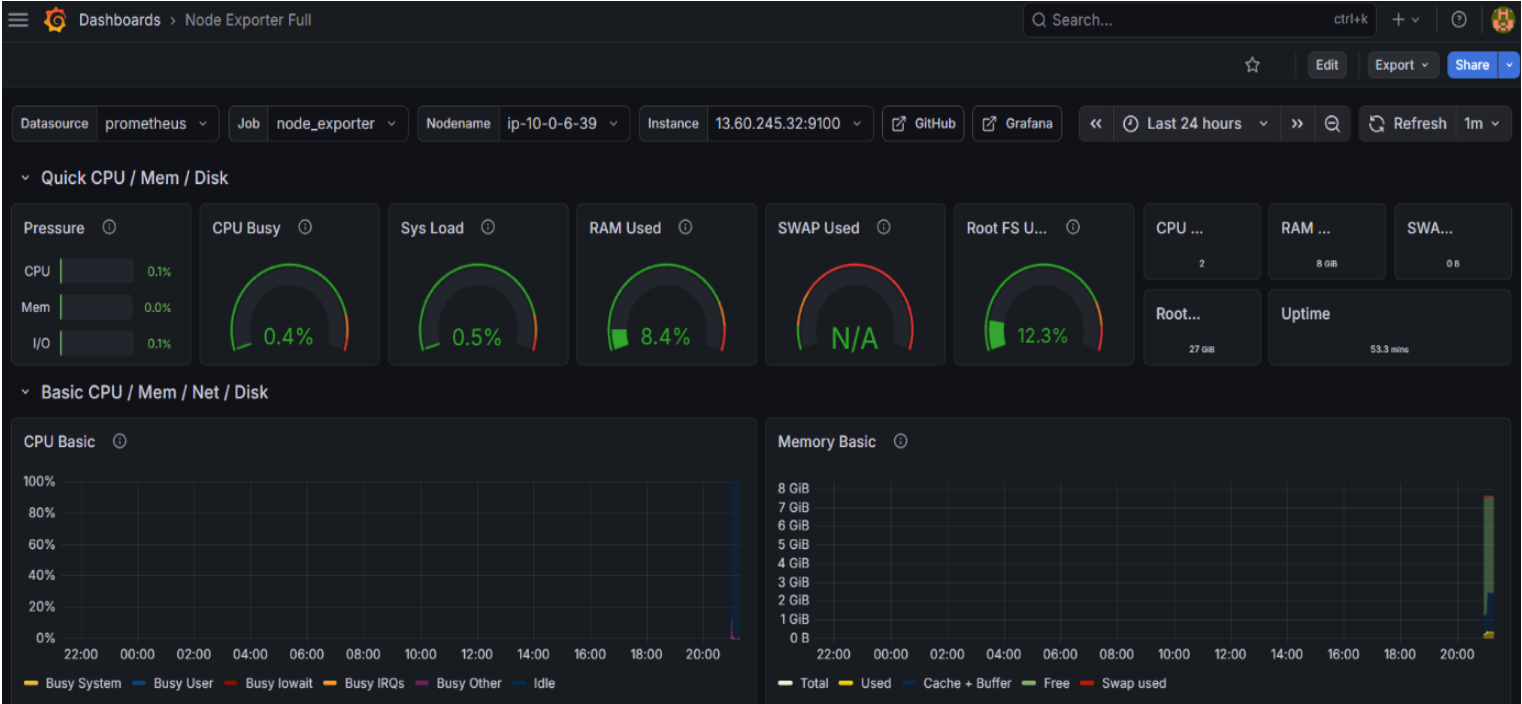
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

rYdddlPWk

Change uid

Import

Cancel



Dashboard: Import dashboard

Import dashboard

Import dashboard from file or Grafana.com

Upload dashboard JSON file

Drag and drop here or click to browse

Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

9964 Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_0HnEoN4z",
  "panels": [...],
  ...
}
```

Load Cancel

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](https://grafana.com)

Published by haryan

Updated on 2023-08-24 15:04:53

Options

Name

Jenkins: Performance and Health Overview

Folder

📁 Dashboards

Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used to uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

haryan-jenkins

Change uid

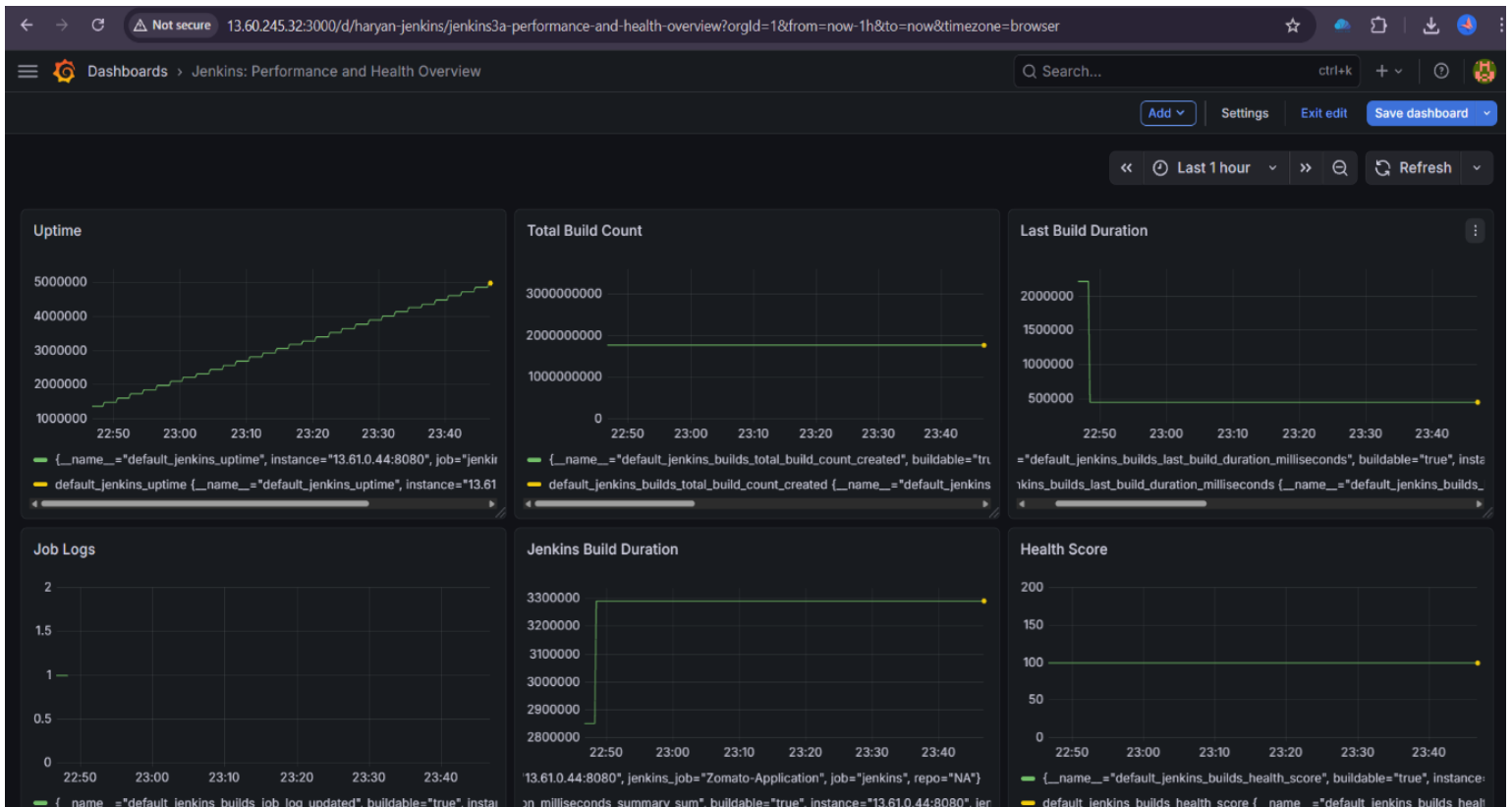
DS_PROMETHEUS

Select a prometheus data source



Import

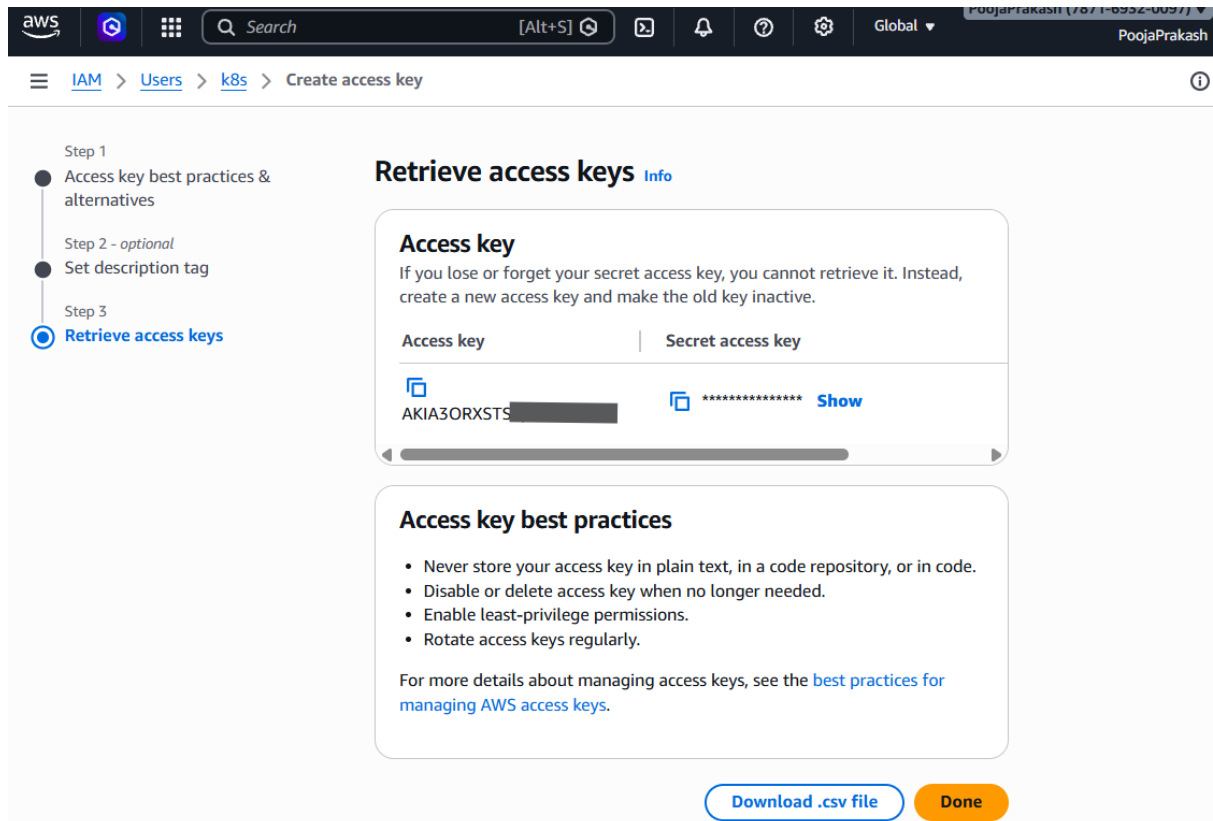
Cancel



Phase 3: Kubernetes Deployment using Amazon EKS

1. EKS Cluster Setup

- Generated AWS Access Key and Secret Key from IAM user and configured AWS CLI in VS Code.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [X] ... [ ] [X] X
● PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> whoami /groups | find
str "S-1-16-12288"
Mandatory Label\High Mandatory Level Label S-1-16-12288
● PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> aws configure
AWS Access Key ID [*****OEXW]: AKIA3ORXSTSQ5MCZOEXW
AWS Secret Access Key [*****6FR7]: 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
Default region name [eu-north-1]: eu-north-1
Default output format [json]: json
○ PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> |
```

- Installed kubectl and eksctl

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [X] ... [ ]
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl version --client
Client Version: v1.28.2
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl version
0.223.0
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> |
```

- Created an EKS cluster named zomatocluster in the same region and VPC.

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl create cluster --name=zom
atocluster --region=eu-north-1 --vpc-public-subnets=subnet-02c17cda1d166a85b,subnet-061485a8046389ab0 --without-nodegroup
```

- Verified cluster and CloudFormation stacks in AWS Console.

aws [] [] [] Search [Alt+S] [] [] [] [] [] Europe (Stockhol PoojaPrakash (7871-6932-0097) PoojaPrakash

Amazon Elastic Kubernetes Service > Clusters > zomatocluster ⓘ

zomatocluster

⌂ Delete cluster Upgrade version Monitor cluster

ⓘ ▶ End of standard support for Kubernetes version 1.34 is December 2, 2026. Upgrade

▼ Cluster info ⓘ

Status ✓ Active	Kubernetes version ⓘ 1.34	Support period ⓘ Standard support until December 2, 2026	Provider EKS
Cluster health ✓ 0	Upgrade insights ✓ 5	Node health issues ✓ 0	Capability issues ✓ 0

< Overview Resources Compute Networking Add-ons 1 Capabilities Access Ob >

Details

API server endpoint https://EFFF36A8196B144BBF8F7F079016347A.gr7.eu-north-1.eks.amazonaws.com	OpenID Connect provider URL https://oidc.eks.eu-north-1.amazonaws.com/id/EFFF36A8196B144BBF8F7F079016347A	Created 22 minutes ago Cluster ARN
--	--	--

CloudFormation > Stacks

Stacks (1)

Search by stack name

Filter status: Active View nested

Stack name	Status	Created time	Description
eksctl-zomatocluster-cluster	CREATE_COMPLETE	2026-02-25 01:39:29 UTC+0530	EKS cluster (dedicated VPC: false, dedicated IAM: true) [created and managed by eksctl]

- Associated OIDC identity provider to enable IAM roles for service accounts.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl utils associate-iam-oidc-provider --region eu-north-1 --cluster zomatocluster --approve
2026-02-25 02:17:01 [i] will create IAM Open ID Connect provider for cluster "zomatocluster" in "eu-north-1"
2026-02-25 02:17:03 [✓] created IAM Open ID Connect provider for cluster "zomatocluster" in "eu-north-1"
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

- Created a Node Group in public subnets with required add-ons.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> eksctl create nodegroup --cluster=zomatocluster --region=eu-north-1 --name=zomato-project-ng --node-type=t3.small --nodes=2 --nodes-min=2 --nodes-max=4 --node-volume-size=20 --ssh-access --ssh-public-key=linux1 --managed --asg-access --external-dns-access --full-ecr-access --appmesh-access --alb-ingress-access
```

- Verified node group and worker nodes.

Amazon Elastic Kubernetes Service > Clusters > zomatocluster

Node groups (1) Info

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Filter node groups by property or value

Group name	Desired size	AMI release version	Launch template
zomato-project-ng	2	1.34.3-20260209	eksctl-zomatocluster-nodegroup-zomato-proje

☰ [EC2](#) > Instances 🔍 📄

Instances (4) Info

Last updated  less than a minute ago

[Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

🔍 Find Instance by attribute or tag (case-sensitive) Running < 1 > 

☐	Name 	Instance ID	Instance state	Instance type
☐	zomatocluster-zomato-project-ng-Node	i-01aaeb4e835eb376e	 Running  	t3.small
☐	Zomato Server	i-0ff1b4044aacd2c45	 Running  	m7i-flex.large
☐	Monitoring Server	i-04e82ae387f0e5c6e	 Running  	m7i-flex.large
☐	zomatocluster-zomato-project-ng-Node	i-0a0c71e9bb2b7c53a	 Running  	t3.small

2. Kubernetes Deployment

- Created Kubernetes YAML files for deployment and service.

📄 master [DevOps-Project-Zomato-Kastro](#) / [Kubernetes](#) / [deployment.yaml](#)

 PoojaPrakash08 Update deployment.yaml

Code Blame

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: zomato
5    labels:
6      app: zomato
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: zomato
12   template:
13     metadata:
14       labels:
15         app: zomato
16     spec:
17       containers:
18       - name: zomato
19         image: poojaprakash08/zomato:latest
20       ports:
21       - containerPort: 3000
```



master ▾

DevOps-Project-Zomato-Kastro / Kubernetes / node-service.yaml



KastroVKiran Add files via upload

Code

Blame

```
1  kind: Service
2  apiVersion: v1
3  metadata:
4    name: node-exporter
5    namespace: prometheus-node-exporter
6  spec:
7    selector:
8      app: node-exporter
9    ports:
10     - name: node-exporter
11       protocol: TCP
12       port: 9100
13       targetPort: 9100
14     type: NodePort
```



master ▾

DevOps-Project-Zomato-Kastro / Kubernetes / service.yaml



KastroVKiran Update service.yaml

Code

Blame

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: zomato
5    labels:
6      app: zomato
7  spec:
8    type: NodePort
9    ports:
10     - port: 3000
11       targetPort: 3000
12       nodePort: 30001
13    selector:
14      app: zomato
```

- Installed Argo CD for GitOps-based deployment.

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl create namespace argocd
namespace/argocd created
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/v2.4.7/manifests/install.yaml
```

- Exposed Argo CD server publicly.

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl edit svc argocd-server -n argocd
service/argocd-server edited
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get svc argocd-server -n argocd
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
argocd-server	LoadBalancer	172.20.22.120	a1530b493f36c4d2289a6791c45fd47c-1170830466.eu-north-1.elb.amazonaws.com
	80:30966/TCP,443:32133/TCP	8m6s	

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get ns
```

NAME	STATUS	AGE
argocd	Active	12m
default	Active	67m
kube-node-lease	Active	67m
kube-public	Active	67m
kube-system	Active	67m

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get pods -n argocd
```

NAME	READY	STATUS	RESTARTS	AGE
argocd-application-controller-0	1/1	Running	0	11m
argocd-applicationset-controller-696ccc6458-ccwmk	1/1	Running	0	11m
argocd-dex-server-7868949c59-h6jvf	1/1	Running	0	11m
argocd-notifications-controller-65b84457bb-4fntd	1/1	Running	0	11m
argocd-redis-868cb656f8-2jgts	1/1	Running	0	11m
argocd-repo-server-6c6b5c6c78-mspht	1/1	Running	0	11m
argocd-server-9c88f7cf9-bw6lc	1/1	Running	0	11m

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

- Installed Node Exporter using Helm for Kubernetes monitoring.

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> choco --version
2.6.0
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> choco install kubernetes-helm -y
Chocolatey v2.6.0
Installing the following packages:
kubernetes-helm
```

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm version
version.BuildInfo{Version:"v4.1.0", GitCommit:"4553a0a96e5205595079b6757236cc6f969ed1b9", GitTreeState:"clean", GoVersion:"go1.25.6", KubeClientVersion:"v1.35"}
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl create namespace prometheus-node-exporter
namespace/prometheus-node-exporter created
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> helm install prometheus-node-exporter prometheus-community/prometheus-node-exporter --namespace prometheus-node-exporter
NAME: prometheus-node-exporter
LAST DEPLOYED: Wed Feb 25 03:00:19 2026
NAMESPACE: prometheus-node-exporter
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
1. Get the application URL by running these commands:
  export POD_NAME=$(kubectl get pods --namespace prometheus-node-exporter -l "app.kubernetes.io/name=prometheus-node-exporter,app.kubernetes.io/instance=prometheus-node-exporter" -o jsonpath="{.items[0].metadata.name}")
  echo "Visit http://127.0.0.1:9100 to use your application"
  kubectl port-forward --namespace prometheus-node-exporter $POD_NAME 9100
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application>
```

```
PS C:\Users\POOJA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
aws-node-7jp4q	2/2	Running	0	43m
aws-node-9m2z9	2/2	Running	0	43m
coredns-65cf8c5645-8np9t	1/1	Running	0	84m
coredns-65cf8c5645-w297z	1/1	Running	0	84m
kube-proxy-6bts4	1/1	Running	0	43m
kube-proxy-h4t7h	1/1	Running	0	43m
metrics-server-b59777598-8dvxv	1/1	Running	0	82m
metrics-server-b59777598-f8v9j	1/1	Running	0	82m

- Updated prometheus.yml to scrape Kubernetes metrics.

```
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['13.60.245.32:9100']

  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['13.61.0.44:8080']

  - job_name: 'k8s'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['13.53.235.166:9100']

-- INSERT --
```

43,40

```
root@ip-10-0-6-39:~# sudo vi /etc/prometheus/prometheus.yml
root@ip-10-0-6-39:~# promtool check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax
root@ip-10-0-6-39:~#
```

- Verified all targets were up in Prometheus.

The screenshot shows the Prometheus web interface at the URL `13.60.245.32:9090/targets?search=`. The page title is "Targets". Below the title, there are filters: "All scrape pools" (dropdown), "All" (selected), "Unhealthy", and "Expand All". There is also a search bar with the text "Filter by end". The main content area lists four targets, each with a status of "(1/1 up)" and a "show more" button:

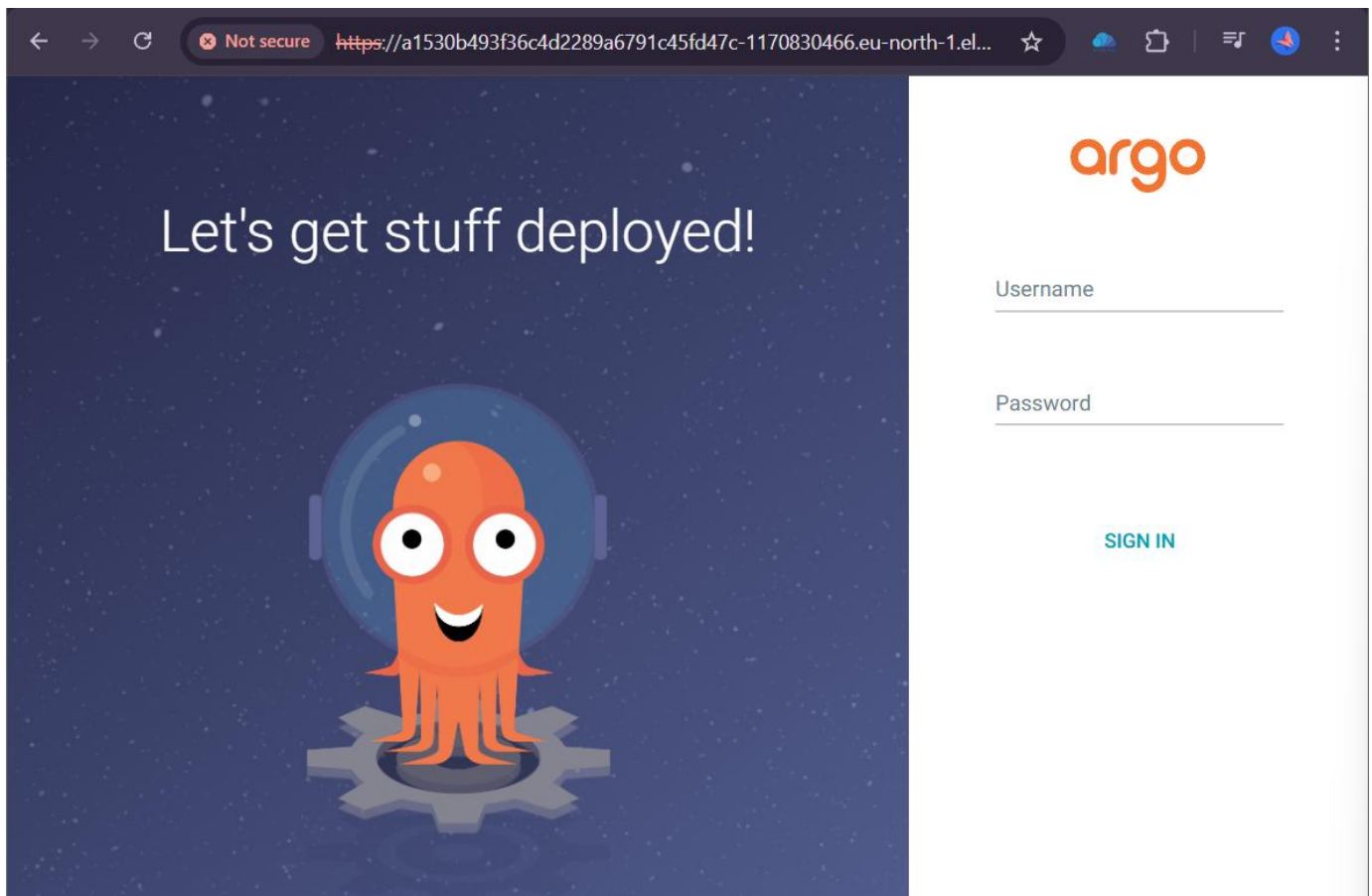
- jenkins (1/1 up)** show more
- k8s (1/1 up)** show more
- node_exporter (1/1 up)** show more
- prometheus (1/1 up)** show more

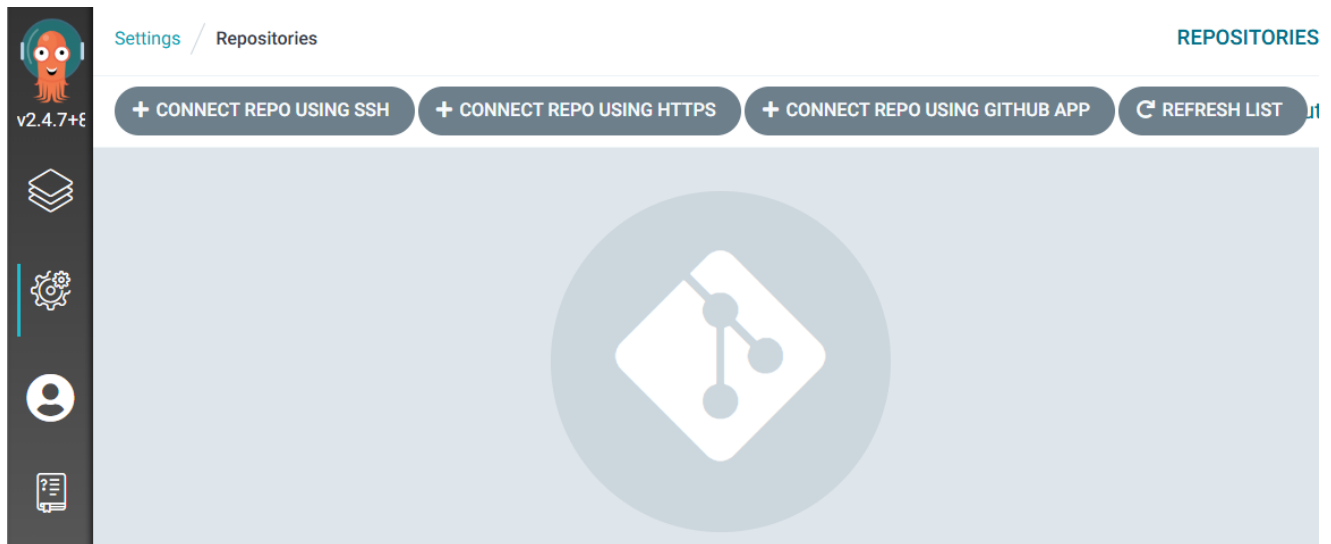
3. Argo CD Application Deployment

- Accessed Argo CD using Load Balancer URL and logged in.

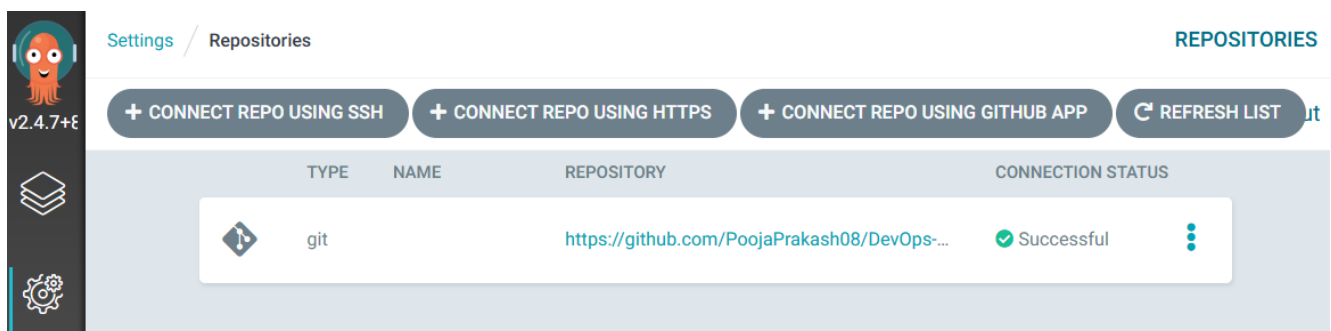
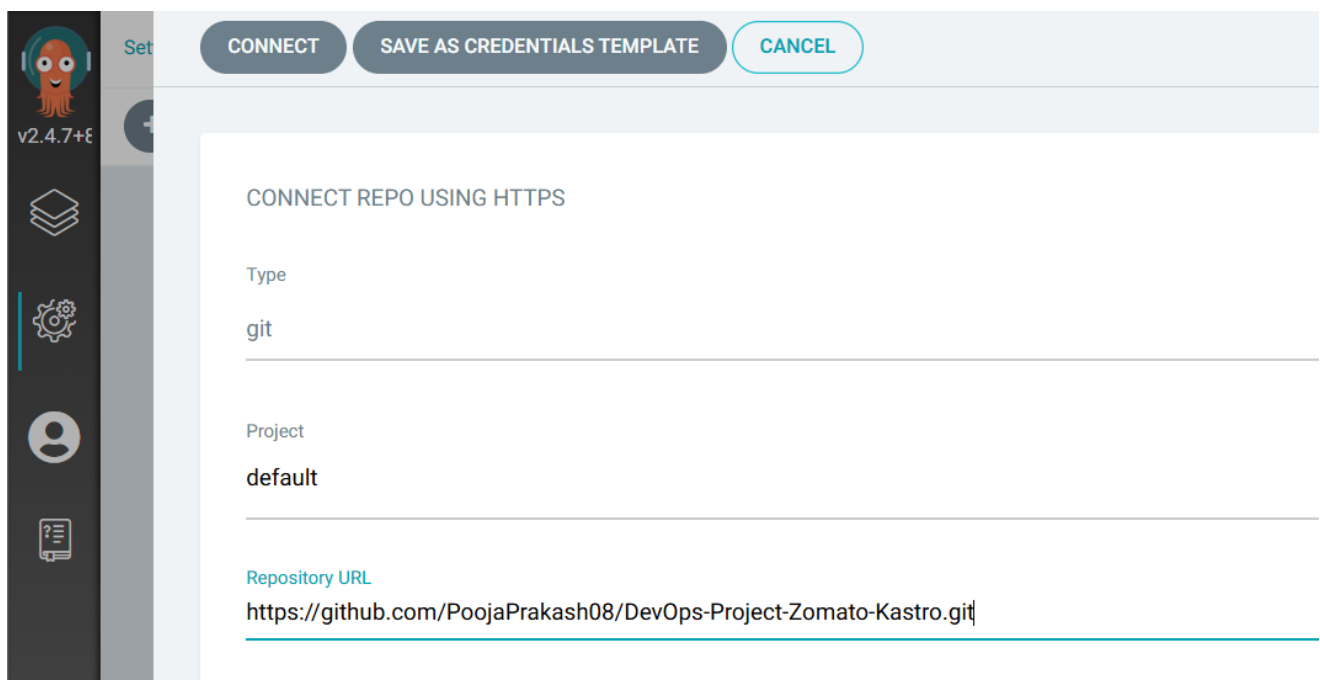
```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> $ARGOCD_SERVER = (kubectl get svc argocd-server -n argocd -o json | ConvertFrom-Json).status.loadBalancer.ingress[0].hostname
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> $ARGOCD_SERVER
a1530b493f36c4d2289a6791c45fd47c-1170830466.eu-north-1.elb.amazonaws.com
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> |
```

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> [System.Text.Encoding]::UTF8
.GetString(
>> [System.Convert]::FromBase64String(
>> (kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}")
>> )
>> )
QrOVNAH-Ghc7Qjby
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Zomato Application> |
```

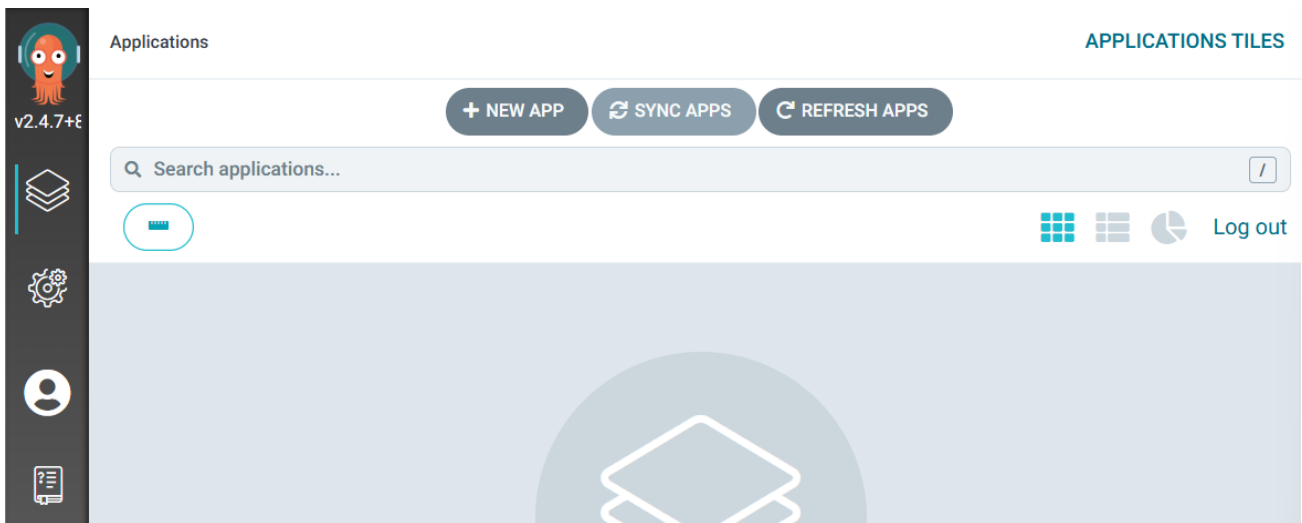




- Connect to the Zomato application repository in Argo CD.



- Add the Zomato application in the argocd



CREATE

CANCEL

×

EDIT AS YAML

GENERAL

Application Name
zomato

Project Name
default

SYNC POLICY

Automatic

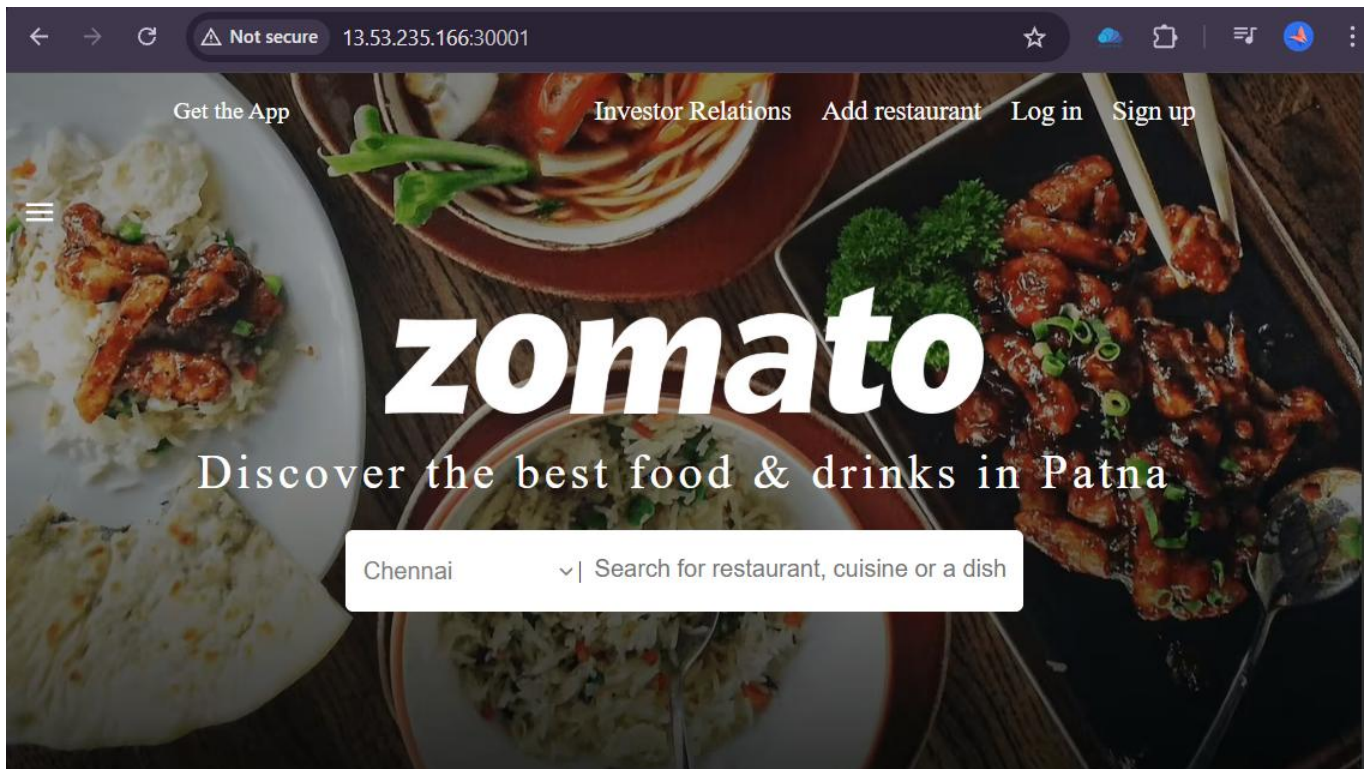
☐ PRUNE RESOURCES ?

☐ SELF HEAL ?

- Monitored application health and sync status from Argo CD dashboard.

The screenshot shows the Argo CD dashboard for the 'zomato' application. The top navigation bar includes 'Applications / Q zomato' and 'APPLICATION DETAILS TREE'. Below this is a toolbar with buttons: 'APP DETAILS', 'APP DIFF', 'SYNC', 'SYNC STATUS', 'HISTORY AND ROLLBACK', 'DELETE', and 'REFRESH'. The main content area displays the application's health and sync status. On the left, there are filters for NAME, KINDS, SYNC STATUS, and HEALTH STATUS. The central part of the dashboard shows a tree view of the application's components, including 'zomato' (svc), 'node-exporter' (svc), and 'zomato' (deploy). Each component has a sync status and a last sync time. The right side of the dashboard shows the application's details, including the 'zomato' service, 'node-exporter' service, and 'zomato' deployment. The 'zomato' service is shown with a sync status of 'Synced' and a last sync time of '16 minutes'. The 'node-exporter' service is also shown with a sync status of 'Synced' and a last sync time of '16 minutes'. The 'zomato' deployment is shown with a sync status of 'Synced' and a last sync time of '16 minutes'. The deployment is also shown with a sync status of 'Synced' and a last sync time of '16 minutes'.

- Accessed the Kubernetes-deployed Zomato application using the Node public IP and NodePort (30001).



- ✓ This project demonstrates a complete end-to-end DevOps implementation covering CI/CD automation, containerization, security scanning, Kubernetes orchestration, GitOps deployment and real-time monitoring. By integrating Jenkins, SonarQube, Docker, Trivy, Prometheus, Grafana, Amazon EKS and Argo CD, the application lifecycle was fully automated from source code to production deployment. This implementation reflects modern DevOps best practices and provides a scalable, secure and observable cloud-native deployment architecture.